Code availability is not enough: schemas for documenting and extending data analysis

H. Sherry Zhang[1], Roger D. Peng[1]

ARTICLE HISTORY
Compiled September 20, 2026

[1] Department of Statistics and Data Science, University of Texas at Austin, Texas, United States

ABSTRACT
Current reproducibility practices have greatly improved our ability to replicate published results, but they do not necessarily make data analyses easier to understand. Readers seeking to comprehend an analysis need to piece together information scattered across manuscripts and code scripts to understand the sequence of analytical steps and the rationale behind them, with some details not explicitly recorded in either source. Exploring alternative analytical choices is even more challenging, as it requires understanding and modifying unfamiliar code, and considering how a change would propagate upstream and downstream. To address these challenges, we propose a schema that represents data analyses as a collection of analytical decisions, together with their rationale, inputs, and outputs. Coupled with large language models (LLMs), the schema can be inferred from natural language descriptions and translated into analysis scripts. The structured representation also enables exploration of alternative analytical choices, allowing multiple analytical workflows to be generated and compared. We demonstrate the proposed workflow in two applications. In the first, we construct a schema for a published regression analysis of heaped smoking data and show that it reproduces the reported estimates, and that a reviewer's suggested alternative, recorded in that paper's supplementary material, can be evaluated by a reader without modifying the original code. In the second, where no schema is supplied, the workflow becomes a reproducibility test: applied to the 29 team reports of the many-analysts football study, it extracts a schema from each report, generates and validates the corresponding code, and compares the result against the published estimate, separating reports that cannot be resolved to a single analysis from failures of the workflow itself. As an intermediate representation between the manuscript and code scripts, our schema gives a self-contained account of an analysis and makes it extensible from a reader's perspective.

KEYWORDS
Reproducibility; Large Language Models (LLMs); Data analysis decisions; Many-analysts study; Statistical workflow

CONTACT: H. Sherry Zhang. Email: hsherryzhang@utexas.edu.

1. Introduction

When we want to understand a published data analysis, beyond simply being able to re-run it, there is currently nothing we can use directly for this purpose. Code and data availability, now standard practice following a broader push for reproducible research (Peng, 2011; Wrobel et al., 2024), guarantee that a result can be regenerated, but they say nothing about whether a reader can track the decisions the authors made, understand the rationale behind them, or comfortably modify the code to explore alternative choices.

Part of the difficulty is that code and the applied researchers who write it operate at different levels: code specifies precisely what a computer should do, while a researcher reasons about what decision that computation represents in the context of their analysis, a gap also identified in prior work on authoring statistical models from conceptual relationships (Jun, Seo, et al., 2022; Jun, Birchfield, et al., 2022). Consider a line such as `dplyr::filter(df, x > 0)`, which has a precise, unambiguous meaning as code: keep rows where `x` is positive. As an analytical decision, however, that same line could represent restricting the analysis to a subpopulation for which `x > 0` is substantively meaningful, or excluding observations flagged as invalid by an encoding convention where $x \leq 0$ denotes missingness; nothing in the `filter()` syntax gives any indication of which of these two decisions is being made. Tools such as `tidyverse` (Wickham et al., 2019) have worked to align code more closely with the action being performed, naming operations after what they do rather than how they are implemented. But knowing the action a line of code performs is not the same as knowing the decision it encodes: an analysis requires additional context that the action alone does not carry.

This suggests that a more effective representation of an analysis would be one that describes the decisions made and their rationale, allowing readers to understand the analysis without needing to parse the code itself or reconstruct the rationale from the manuscript. Recent advances in Large Language Models (LLMs) for code generation make such a representation practical: LLMs are now widely used for code generation tasks, demonstrating strong capabilities in translating natural language descriptions into executable programs (Chen et al., 2021; Huang et al., 2022; Jiang et al., 2026; Lai et al., 2023; Shah et al., 2026). Once a decision is described in a schema, an LLM can generate and validate the corresponding code directly from it.

This same representation also makes extending an analysis easier. Data analysis is often underdetermined, with many plausible choices regarding data preprocessing, transformation, aggregation, and model specification (Bertran et al., 2026; Gao & Xiao, 2026; Luo et al., 2025), and a reader may reasonably want to know whether a conclusion would still hold under one of these other choices, for example, "What if we had included observations with $x \leq 0$?" This question motivates multiverse analysis (Steegen et al., 2016) and specification curve analysis (Simonsohn et al., 2020), which report a result across the set of defensible analytical choices rather than a single path through it. Because each decision is recorded explicitly rather than embedded in code, a reader can

specify such an alternative directly and have it evaluated through the same schema and the same LLM-based translation into code, rather than modifying the original script by hand.

In this paper, we present a schema designed to describe data analysis in a structured way that captures the decisions and their rationale, as well as the inputs and outputs of each step. We also demonstrate how this schema, coupled with LLMs, can be used to generate code scripts from natural-language descriptions in manuscripts, and explore alternative decision choices. We present two applications. The first takes the perspective of an author supplying a schema alongside a published manuscript, using the GAITD regression analysis of heaped smoking data in Yee et al. (2025): the schema reproduces the published estimates exactly, and a reviewer's suggestion recorded in that paper's supplementary material is then evaluated as an alternative file, without modifying the original analysis script. The second takes the opposite situation, where no schema is supplied and the workflow serves instead as a reproducibility test: applied to the many-analysts football study, it extracts a schema from each team's report, generates and validates the corresponding code, and compares the result against that team's published estimate. The comparison separates reports whose description cannot be resolved to a single analysis from failures of the workflow itself, and in the process uncovers an inconsistency in one team's published report between the covariate described in the text and the one implied by their results. We further demonstrate how the schema supports specification and exploration of alternative random effect structures for Team 5.

The paper is organized as follows: In Section 2, we present our methodology, including the schema representation for describing data analysis and the workflow for generating and validating code from the schema by LLMs. In Section 3, we construct a schema for a published GAITD regression analysis and show how it allows a reader to evaluate an alternative specification directly. In Section 4, we apply the workflow as a reproducibility test to the many-analysts football study, recovering the reported results for most teams and using the discrepancies to separate inconsistencies in the original reports from failure modes of current LLMs. Finally, in Section 5, we discuss the implications of our work and potential future directions.

2. Methodology

As traditional reproducibility practices can be insufficient for understanding complex modern data analyses, we have designed two complementary YAML formats and the associated workflow to better support comprehension and extensibility in applied data analysis. Our workflow is designed to be 1) decision-centric: analyses are represented as a sequence of decisions with explicit rationale, and 2) reader-oriented exploration: readers can inspect, compare, and extend those decisions.

2.1. *A schema for describing data analysis*

We represent a data analysis, $\mathcal{S} = \{A_1, A_2, \cdots, A_n\}$, as a sequence of n analytical steps A_i, applied to a dataset $\mathcal{D}$, where each step, A_i, is defined as a tuple:

$$A_i = \langle f_i, \pi_i, \rho_i, \mathcal{I}_i, \mathcal{O}_i, \omega_i \rangle. \tag{1}$$

The components of A_i can be partitioned into two parts: a required triplet (f_i, π_i, ρ_i) and an optional triplet $(\mathcal{I}_i, \mathcal{O}_i, \omega_i)$. The first component of the triplet, $f_i \in \mathcal{F}$, is the *objective* of a step, identifying the problem the step is trying to solve and thereby fixing the space of alternative decisions that could equally address it (formally introduced in Section 2.2). The second component, $\pi_i \in \Pi$, is the *decision* made to achieve f_i: it is a narration, in natural language, of what a corresponding line of code would do, situated in the context of this specific analysis, rather than the code's syntax itself. The term ρ_i is the *rationale* provided for choosing the decision π_i over the other alternatives that could have fulfilled the same objective; it is recorded in the `rationale` field of the schema. Structuring this information into a schema involves defining the three components described above for each step. As an example, consider the description for calculating the Human Development Index (HDI) (United Nations Development Programme, 2025):

1) Scale all four variables (life expectancy, expected years of schooling, mean years of schooling, and GNI per capita) to the [0, 1] range.
2) Combine the two education variables (expected years of schooling and mean years of schooling) into a single education variable.
3) Combine the three dimensions (health, education, and income) into a single index using a geometric mean.

In Step 3, the objective is to combine the three dimensions. While a simple arithmetic mean could be an alternative decision to fulfill this goal, the geometric mean (the decision) is chosen because it penalizes uneven development among the three dimensions (the rationale). The full schema for constructing HDI is presented in Listing 1.

Listing 1 An example schema for calculating Human Development Index (HDI)

```yaml
meta:
  type: schema
nodes:
- id: step-scaling
  objective: how to handle variables in different scales?
  decision: apply min-max scaling to each variable
  rationale: to convert the variables to the same scale for future
    construction of a single index
- id: step-education
  objective: how to combine the two education-related variables?
  decision: take the average
```

```
  rationale: the arithmetic mean allows perfect substitutability
    between the two variables
- id: step-combine
  objective: how to combine the three dimensions into a single index?
  decision: use the geometric mean
  rationale: the geometric mean penalizes uneven development
    among the three core dimensions
```

The required triplet (f_i, π_i, ρ_i) is already sufficient on its own for a human reader to follow the analytical logic of a step, without needing access to the underlying dataset. The optional triplet $(\mathcal{I}_i, \mathcal{O}_i, \omega_i)$ adds a further layer on top of this: $\mathcal{I}_i$ and $\mathcal{O}_i$ bind the step to concrete variables in the dataset, resolving ahead of time the variable mapping that an LLM would otherwise have to infer at the point of code generation. Populating them requires a dataset to be available, which is why they remain optional rather than required. Because the schema is meant to specify an analysis before the corresponding code is generated, the output of a step can name a variable that does not yet exist in the dataset; this is inherent to using the schema as a specification rather than a post-hoc description of existing code. The inputs for any given step, by contrast, must be within the existing data scope, comprising variables in the original dataset and all outputs from previous steps. In practice, $\mathcal{O}_i$ most often names a column added to the evolving dataset via non-standard evaluation, as in `tidyverse`-style pipelines; a small number of steps, such as model fitting, instead produce a distinct object that is not itself a column of $\mathcal{D}$. Formally, we require:

$$\mathcal{I}_i \subseteq \left(\mathcal{D} \cup \bigcup_{j<i} \mathcal{O}_j \right), \quad \forall i \in \{2, \dots, n\}.$$

When the schema is extracted automatically by Large Language Models (LLMs) from text, the inputs and outputs will be inferred by the LLMs, if a dataset is provided. We also introduce a categorical confidence score, ω_i (low, medium, high), defined as the model's confidence that the recorded (f_i, π_i, ρ_i) correctly reflects the source text; ω_i is low precisely when one or more of these components could not be read off the text directly and had to be inferred by the LLM. Unlike the other components, ω_i is not part of the account of the analysis that a reader consumes; it is consumed by the implementation, which uses it to flag steps for the author to review or to prompt for clarification before the schema is treated as settled.

Recording a step's decision π_i does not mean it is the only one that could satisfy its objective f_i: a reader may want to revisit the decision made for some of these steps and consider alternatives that address the same objective, a possibility we formalize next.

2.2. *Describing alternatives*

Another analyst following the HDI schema in Listing 1 might wonder whether the reported index would look different under a different decision at some step. Enumerating such choices is the premise of multiverse analysis, and tooling exists to support it: the `multiverse` package, for instance, lets an analyst declare alternative branches inside an R notebook and expands them into the full set of analyses (Sarma et al., 2023). What that tooling requires is code — each branch is expressed in the language the analysis is written in, by someone able to write it. Our alternatives are specified at the level of decisions instead, in natural language, and the code for each is generated afterwards. The difference matters most for who can pose the question: a multiverse is normally built by the original analyst while exploring their own garden of forking paths, whereas an alternative file can be written by a reader after publication, against a schema the author has already supplied. Formally, a step in the schema can have a candidate set of alternatives: step i may admit k_i decisions that all satisfy its objective f_i, $\{A_i^1, \dots, A_i^{k_i}\}$, of which the decision π_i recorded in the schema is just one member, A_i^1. An alternative file specifies the remaining candidates, $A_i^2, \dots, A_i^{k_i}$, for a subset of steps $C \subseteq \{1, \dots, n\}$.

The overall structure of an alternative file is shown in Listing 2. The `meta` field at the top identifies the file as an alternative specification (`type: alternatives`), and the file-level `branch` field, either `single` or `multi`, determines how the candidates at each step combine, described below. The file itself is organized as a list, one entry per step in C (`nodes` in the YAML): each entry names its step in the original schema (via `overrides`) and lists the alternative decisions for that step, together with their rationale.

Listing 2 The skeleton of an alternative YAML file, which consists of a `branch` mode and a list of covered steps, each specifying a step in the original schema and its alternative decisions.

```yaml
meta:
  type: alternatives
  branch: single | multi
nodes:
  - overrides: <step id>
    alternatives:
      - id: <alternative id>
        decision: <...>
        rationale: <...>
      - id: <alternative id>
        ...
  - overrides: <step id>
    alternatives:
      ...
  ...
```

Under `branch: single`, every step is restricted to a single candidate decision. These alternatives and the unchanged steps in the original schema together form one new anal-

ysis. This is useful when an analyst wants to see what a different, internally consistent analysis touching several steps at once would look like. Listing 3 fills in the skeleton to construct such an alternative version of the HDI computation, using a harmonic mean to combine the two education variables in Step 2 and an arithmetic mean to combine the three dimensions in Step 3.

Listing 3 A sample alternative file that changes the decisions for Step 2 and Step 3 in calculating Human Development Index (HDI) to use a harmonic mean and an arithmetic mean, respectively.

```yaml
meta:
  type: alternatives
  branch: single
nodes:
  - overrides: step-education
    alternatives:
      - id: harmonic-mean
        decision: take the harmonic mean of the two education variables
        rationale: penalizes disparity between the two schooling measures
          rather than allowing one to compensate for the other
  - overrides: step-combine
    alternatives:
      - id: arith-mean
        decision: use an arithmetic mean
        rationale: a simple average of the three dimensions
```

Sometimes a reader instead wants to test a range of alternatives for one or more steps independently. Under `branch: multi`, the candidates at each covered step, along with that step's original decision, are crossed to produce every combination. In Listing 4, one alternative is specified for the education step (the harmonic mean) and two for the combination step (an arithmetic mean and the minimum of the three dimensions). Together with the original schema, this produces $(1 + 1) \times (2 + 1) = 6$ analyses. More generally, with k_i candidates at each step $i \in C$, crossing all candidates yields $\prod_{i \in C} k_i$ analyses.

Listing 4 A sample alternative file that explores one alternative decision for Step 2 and two for Step 3 in calculating Human Development Index (HDI). Combining these alternatives with the original schema produces a total of six $(1+1) \times (2+1) = 6$ analyses.

```yaml
meta:
  type: alternatives
  branch: multi
nodes:
  - overrides: step-education
    alternatives:
      - id: harmonic-mean
        decision: take the harmonic mean of the two education variables
        rationale: penalizes disparity between the two schooling measures
```

```
        rather than allowing one to compensate for the other
- overrides: step-combine
  alternatives:
    - id: arith-mean
      decision: use an arithmetic mean
      rationale: a simple average of the three dimensions
    - id: min-agg
      decision: use the minimum of the three dimensions
      rationale: a "weakest-link" aggregation that penalizes any single
        lagging dimension more strongly than an average would
```

2.3. *Workflow*

The overall workflow for schema-based data analysis is illustrated in Figure 1. A schema can be created either manually from the template or automatically by an LLM from a natural-language manuscript. Once created, the schema serves as a structured specification that guides a second LLM in generating the code for the analysis. While the latest LLMs can generate reasonable code from a prompt, they do not consistently achieve perfect accuracy. To improve reliability, our design includes a third LLM agent responsible for code validation. The validation process runs the generated script in a sandbox environment and, if an error occurs, this agent is employed to diagnose the script. The validation agent receives the generated code together with the error message and the corresponding line number produced during the execution, and returns a newer version of the script. The revised script is then re-executed in the sandbox and this cycle continues until one of the three conditions is met: the code runs successfully, a maximum number of attempts is reached, or the user terminates the process. Validation in this sense establishes that the script executes to completion without error; it does not establish that the code correctly implements the decisions recorded in the schema, and a script can therefore pass validation while still departing from the intended analysis. As shown in Section 4, this process resolves the majority of R syntax issues encountered during data wrangling and statistical modelling tasks, though we also encounter cases where it does not. On top of supporting a single analytical specification, the workflow also allows users to augment a schema with an alternative YAML file. This expands the original schema into a multiverse of parallel schemas, each representing a different sequence of decision choices. The same code generation and validation workflow is then applied independently to each schema, producing a corresponding set of validated scripts.

Returning to the HDI example, the proposed workflow allows both authors and readers to interact with a data analysis through natural language. The schema in Listing 1 is straightforward to construct from a template and can be included as supplementary material to document the analytical workflow. An author may use the "code generation and validation" path in Figure 1 to automatically generate the analysis script for computing HDI and validate its accuracy. A reader, on the other hand, may wish to explore

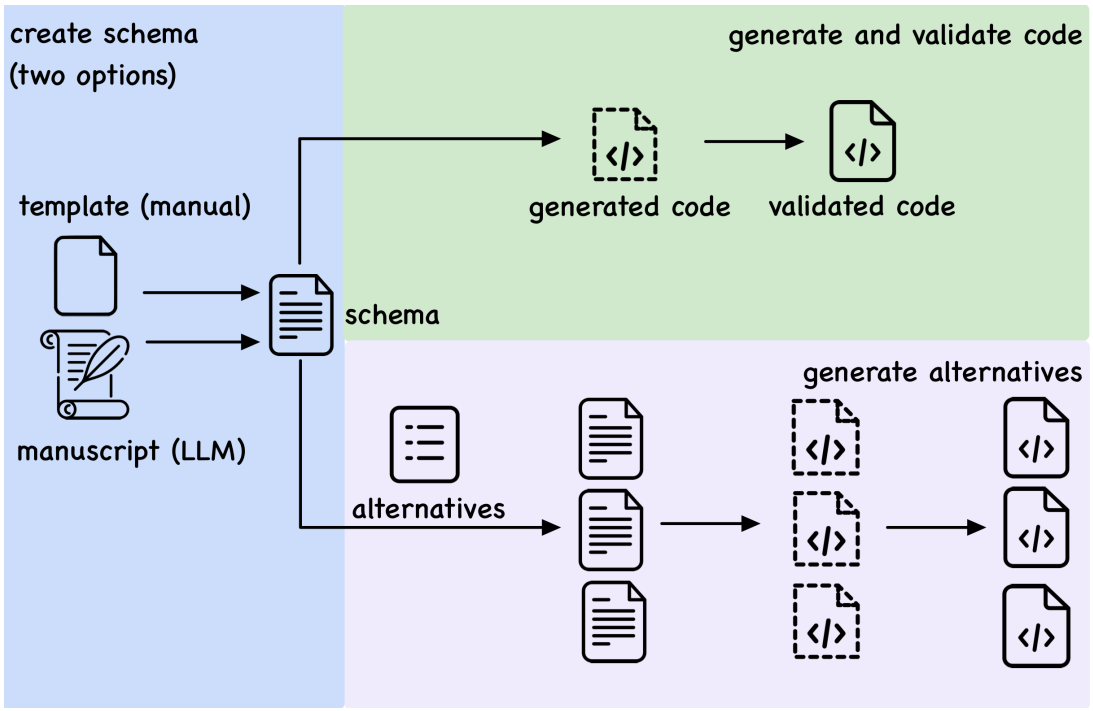


Figure 1. Overview of the schema-driven analysis workflow. Schemas can be authored manually or generated from natural language manuscripts by an LLM. Schemas are used to generate code, which is validated through iterative execution in a sandboxed environment and repaired when errors occur. Alternative specifications can expand a schema into a multiverse of parallel analyses.

how the HDI values or country rankings change if an arithmetic mean is used instead of the original geometric mean. By combining the original schema in Listing 1 with the alternative specification in Listing 4, the "alternative generation" path produces a corresponding analysis script for each alternative with LLMs. In contrast, modifying an existing code script by hand typically requires manually coordinating changes across upstream and downstream code. By representing analyses as a sequence of structured analytical decisions, our workflow allows LLMs to automate much of this process, making it easier to understand the sequence of steps taken in the analysis and to explore alternatives.

3. Application: GAITD regression of heaped smoking data

In this section, we demonstrate the schema workflow from an author's perspective: what it looks like for an author to provide a schema alongside a manuscript, and what that then lets an audience do to question and explore the analysis with much less effort than the manuscript and its code alone would require. We use Yee et al. (2025)'s GAITD regression analysis of heaped smoking data, published in the Annals of Applied Statistics, as the example. Yee et al. (2025) report four separate smoking-related responses; for demonstration purposes we build the schema for only the first, smoking initiation age (P1, Section 5.1 of their paper), and the same workflow extends to the other three.

Listing 5 contains fourteen entries; rather than list them all, we highlight one as representative of what each entry records. Every entry follows the *objective*, *decision*, and *rationale* structure introduced in Section 2.1. Consider `special_value_mid_risk`: the decision is to treat ages 12-14 and 19-21 as one inflated group with extra probability mass, since these ages show a moderate, secondary elevation in the spikeplots, surrounding the sharper peak already attributed to the high-risk group (ages 15-18). Recording the rationale along with the decision allows a reader to agree or disagree with the authors' choice, and to question and explore the analysis, as we do later in this section. The other thirteen entries follow the same structure.

Listing 5 A schema for the GAITD regression analysis of smoking initiation age in Yee et al. (2025), constructed from the manuscript and intended to be provided alongside it as a self-contained account of the analysis.

```
meta:
  type: schema
nodes:
- id: complete_case_filtering
  objective: handle missing data in selected variables
  decision: apply complete-case filtering to all columns except TC30, TCquit,
   ↪ and SCA which have NAs by design
  rationale: TC30, TCquit, and SCA are exempted from the completeness check
   ↪ since they are interlaced with NAs by design (e.g., SCA being NA means
   ↪ the person has not yet quit, a valid state rather than missingness);
   ↪ every other selected column is required to be non-missing because most
   ↪ smoking studies adjust for age, marital status, education and race, and
   ↪ this analysis follows suit.
```

```yaml
- id: age_restriction
  objective: define the age range for analysis
  decision: restrict analysis to respondents aged less than 80 years (age <
   ↪ 80)
  rationale: age was right-censored at 80

- id: data_quality_correction
  objective: handle logically impossible self-reported values
  decision: set both SCA and SIA to NA where SCA - SIA < 0
  rationale: quitting age before smoking initiation age is a logical
   ↪ impossibility and such values cannot be trusted

- id: impute_marital_young
  objective: assign a defensible marital status for young respondents,
   ↪ regardless of what was recorded
  decision: recode marital status to "Never married" for everyone age < 20
  rationale: this age group is effectively certain to have never been
   ↪ married, so the value is set by age rather than trusting the raw survey
   ↪ response

- id: impute_education_young
  objective: assign a defensible education category for young respondents,
   ↪ regardless of what was recorded
  decision: recode education to "< 9th grade" for age < 14; "9-11th grade"
   ↪ for 14 <= age <= 16; "High school" for 17 <= age <= 19
  rationale: for respondents still of school age, highest-grade-completed is
   ↪ essentially determined by age, so the value is set by age rather than
   ↪ trusting the raw survey response

- id: define_ever_smokers_sample
  objective: define the analytic sample of ever-smokers
  decision: include as valid observation if both conditions are met -- 1) SIA
   ↪ is not missing, and 2) either SCA and TCquit are both not missing, or
   ↪ SCA is missing and TC30 is not missing
  rationale: SCA is missing by design for anyone who has not yet quit. the
   ↪ two-branch rule instead admits ex-smokers through the SCA/TCquit branch
   ↪ and current smokers through the TC30 branch

- id: parent_distribution_choice
  objective: select appropriate probability distribution for count response
   ↪ variable
  decision: adopt negative binomial distribution as the parent distribution
  rationale: the SIA distributions iboth unimodal and right-skewed, so a NBD
   ↪ parent is reasonable; more generally, there is evidence of
   ↪ overdispersion relative to a Poisson distribution.

- id: special_value_mid_risk
  objective: specify how to model inflation at ages surrounding peak teenage
   ↪ smoking uptake
  decision: treat ages 12, 13, 14, 19, 20, 21 as a parametrically-inflated
   ↪ group, with the extra probability mass there sharing the same
   ↪ distributional shape (mean and dispersion) as the main population
```

```
  rationale: this mid-risk subgroup represents early and later smokers
   ↪ surrounding the high-risk age subgroup; these ages showed a moderate,
   ↪ secondary elevation in repeated training/test spikeplots of the data

- id: special_value_high_risk
  objective: specify how to model inflation at peak teenage smoking uptake
   ↪ ages
  decision: treat ages 15, 16, 17, 18 as a nonparametrically-inflated group,
   ↪ with each age given its own freely-estimated extra probability
  rationale: this high-risk subgroup corresponds to frenetic teenage smoking
   ↪ uptake, the age range where the largest number of regular smokers
   ↪ start; these ages showed the largest, most persistent spikes across
   ↪ repeated training/test spikeplots of the data.

- id: alteration_operator_decision
  objective: determine whether to apply alteration operators
  decision: do not use alteration operators (A.mix and A.mlm set to NULL)
  rationale: not needed for this model specification

- id: deflation_operator_decision
  objective: determine whether to apply deflation operators
  decision: do not use deflation operators (D.mix and D.mlm set to NULL)
  rationale: not needed for this model specification

- id: truncation_decision
  objective: determine whether to specify a truncated region
  decision: do not use truncation (no truncated region specified for SIA)
  rationale: truncation not appropriate for this response variable

- id: covariate_selection
  objective: select covariates for the linear predictor of parent
   ↪ distribution mean
  decision: include race2, gender, marital, and educ as covariates
  rationale: most smoking studies adjust for age, marital status, education
   ↪ and race

- id: fit_final_model
  objective: fit the final GAITD regression model
  decision: fit a single vglm that puts together the pieces above --
   ↪ response, parent distribution, special-value structure (inflation,
   ↪ alteration, deflation, and truncation), and covariates. The covariates
   ↪ only shift the mean of the negative binomial; the dispersion and the
   ↪ mid-/high-risk inflation probabilities are held fixed across the sample
  rationale: only the mean is modeled with covariates; the dispersion and
   ↪ inflation probabilities are kept intercept-only, since extending
   ↪ covariate effects to them would substantially increase the number of
   ↪ parameters to estimate and make the results harder to interpret
```

Assembling the schema required no analysis the authors had not already done, but it did require collecting the decisions from two documents and, in places, from code

alone. Five of the fourteen appear nowhere in the manuscript. The manuscript records that cases with missing values in the adjustment variables were deleted and that the sample was restricted to respondents under 80; it does not record that `TC30`, `TCquit`, and `SCA` are exempt from that completeness check, that `SCA` and `SIA` are set to missing wherever `SCA - SIA < 0`, that marital status and education are overwritten by age for respondents under 20, or the two-branch rule that defines the ever-smoker sample. Each of these survives only as executable code in the supplementary material, and their stated rationale, where there is one, is a code comment: the completeness exemption is explained by the remarks that the variables are "interlaced with NAs" and that a missing value "means not yet quit, which is okay", the ever-smoker rule by the comment "Complex to handle NAs", and the data-quality correction by the single word "Bad". The two recodings carry no comment at all; a reader must infer their purpose from a note several pages later, written for another purpose, observing that marital status, education, and smoking initiation age "were not measured in those aged 19 or less". Recovering these decisions is exactly the piecing-together the schema is meant to remove, and it is worth emphasising that Yee et al. (2025)'s supplementary material is unusually thorough by the standards of applied statistics: it walks through the analysis in a tutorial style, states general modelling guidelines upfront, and discusses competing options it rejected. Even so, five of the fourteen decisions needed to reproduce the published estimates exist only as code.

As a check that this schema is accurate, generating and validating code from it produces the R script in Listing 6, which reproduces the coefficients Yee et al. (2025) report for smoking initiation age exactly, matching their Table 2. The check establishes more than that our own workflow functions. Because the code was generated from the schema alone, with no access to the manuscript or the authors' script, reproducing the published estimates shows that the schema is *sufficient*: nothing material to the analysis is missing from it. A schema that meets this standard could accompany a manuscript as reproducibility material in its own right, alongside code and data availability, and a reader can rely on it as an accurate account of the analysis it documents.

Listing 6 The R script generated and validated from the schema in Listing 5, annotated with the `id` of the schema step each block implements.

```r
library(VGAMdata)
library(VGAM)
library(dplyr)

data("smqP", package = "VGAMdata")

cols <- c('TC30', 'TCquit', 'SCA', 'educ', 'marital',
          'SIA', 'SEQN', 'age', 'gender', 'race2',
          "passiveSmoke_job", "passiveSmoke_rest",
          "passiveSmoke_bar", "passiveSmoke_car",
          "passiveSmoke_home", "passiveSmoke_other",
          "passiveSmoke_ecigarettes")
```

```r
df <- smqP |> select(all_of(cols))

# Step 1: complete_case_filtering
# Apply complete-case filtering to all columns except TC30, TCquit, and SCA
cols_require_complete <- setdiff(cols, c("TC30", "TCquit", "SCA"))
df_complete <- df |>
  filter(complete.cases(pick(all_of(cols_require_complete))))

# Step 2: age_restriction
# Restrict analysis to respondents aged less than 80 years
df_age_restricted <- df_complete |> filter(age < 80)

# Step 3: data_quality_correction
# Set both SCA and SIA to NA where SCA - SIA < 0
df_age_restricted <- df_age_restricted |>
  mutate(
    SCA_corrected = if_else(!is.na(SCA) & !is.na(SIA) & (SCA - SIA < 0),
      ↪ NA_integer_, SCA),
    SIA_corrected = if_else(!is.na(SCA) & !is.na(SIA) & (SCA - SIA < 0),
      ↪ NA_integer_, SIA)
  )

# Step 4: impute_marital_young
# Recode marital status to "Never married" for everyone age < 20
df_age_restricted <- df_age_restricted |>
  mutate(
    marital_imputed = if_else(age < 20, "Never married",
      ↪ as.character(marital))
  )

# Step 5: impute_education_young
# Recode education based on age for young respondents
df_age_restricted <- df_age_restricted |>
  mutate(
    educ_imputed = case_when(
      age < 14 ~ "< 9th grade",
      age >= 14 & age <= 16 ~ "9-11th grade",
      age >= 17 & age <= 19 ~ "High school",
      TRUE ~ as.character(educ)
    )
  )

# Step 6: define_ever_smokers_sample
# Include if: SIA not missing AND (both SCA and TCquit not missing OR SCA
  ↪ missing and TC30 not missing)
df_ever_smokers <- df_age_restricted |>
  filter(
    !is.na(SIA_corrected) &
    ((!is.na(SCA_corrected) & !is.na(TCquit)) | (is.na(SCA_corrected) &
  ↪ !is.na(TC30)))
  )
```

```r
df_ever_smokers <- droplevels(df_ever_smokers)

# Step 7: parent_distribution_choice
# Adopt negative binomial distribution as the parent distribution
parent_distribution <- negbinomial

# Step 8: special_value_mid_risk
# Ages 12, 13, 14, 19, 20, 21 as parametrically-inflated group (i.mix)
i_mix_spec <- c(12, 13, 14, 19, 20, 21)

# Step 9: special_value_high_risk
# Ages 15, 16, 17, 18 as nonparametrically-inflated group (i.mlm)
i_mlm_spec <- c(15, 16, 17, 18)

# Step 10: alteration_operator_decision
# Do not use alteration operators
a_mix_spec <- NULL
a_mlm_spec <- NULL

# Step 11: deflation_operator_decision
# Do not use deflation operators
d_mix_spec <- NULL
d_mlm_spec <- NULL

# Step 12: truncation_decision
# Do not use truncation
truncation_spec <- NULL

# Step 13: covariate_selection
# Include race2, gender, marital, and educ as covariates
covariate_formula <- SIA_corrected ~ race2 + gender + marital_imputed +
   ↪  educ_imputed

# Step 14: fit_final_model
# Fit a single vglm with response, parent distribution, special-value
   ↪  structure, and covariates
fitted_model <- vglm(
  covariate_formula,
  family = gaitdnbinomial(
    i.mix = i_mix_spec, i.mlm = i_mlm_spec,
    a.mix = a_mix_spec, a.mlm = a_mlm_spec,
    d.mix = d_mix_spec, d.mlm = d_mlm_spec,
    truncate = truncation_spec
  ),
  data = df_ever_smokers
)
```

Beyond confirming the schema's accuracy, this workflow lets a reader write an alternative file directly against the schema and obtain generated, validated code, rather than

going through the authors' original script and modifying it by hand. Yee et al. (2025)'s supplementary material reports exactly this kind of question from a reviewer, who suggested splitting the mid-risk group into two, keeping ages 12-14 as inflated but treating ages 19-21 as altered instead, since that older subgroup shows a qualitatively different, declining pattern of uptake. Listing 7 encodes the reviewer's suggestion as an alternative file, overriding three steps at once: `special_value_mid_risk` narrows the inflated group to ages 12-14, `alteration_operator_decision` turns on an alteration group for ages 19-21, and `covariate_selection` drops to an intercept-only model, matching how the authors themselves computed AIC for this comparison; `branch: single` ties the three overrides into one refit, rather than crossing them into separate branches.

Listing 7 An alternative YAML file specifying a joint change to the mid-risk inflation, alteration, and covariate steps in the GAITD schema, treating ages 19-21 as an alteration group rather than pooling them into the mid-risk inflated group.

```yaml
meta:
  type: alternatives
  branch: single
nodes:
  - overrides: special_value_mid_risk
    alternatives:
      - id: ap1921-special-value
        decision: "treat ages 12, 13, 14 only as a parametrically-inflated
   ↪  group, with the extra probability mass there sharing the same
   ↪  distributional shape (mean and dispersion) as the main population; ages
   ↪  19, 20, 21 are no longer pooled into this group (see the alteration
   ↪  operator decision instead)"
        rationale: "ages 19-21 mark less frenetic, declining smoking uptake
   ↪  as respondents age out of the high-risk window, a qualitatively
   ↪  different pattern from the early uptake at ages 12-14, so they are
   ↪  moved out of this pooled inflated group"
  - overrides: alteration_operator_decision
    alternatives:
      - id: ap1921-alteration
        decision: "use ages 19, 20, 21 as an alteration group (A.mix) instead
   ↪  of leaving alteration operators unused"
        rationale: "ages 19-21 mark less frenetic, declining smoking uptake
   ↪  as respondents age out of the high-risk window, a qualitatively
   ↪  different pattern from the early uptake at ages 12-14, so they are
   ↪  represented as an alteration rather than folded into the same inflated
   ↪  group"
  - overrides: covariate_selection
    alternatives:
      - id: ap1921-covariates
        decision: "do not include any covariates (intercept-only model)"
        rationale: "this comparison is about which special-value structure
   ↪  fits better, so covariates are left out to compare the two structures
   ↪  on equal footing, the same way the special-value structure was first
   ↪  established on an intercept-only model before covariates were added"
```

Generating and validating code from this alternative, without the reader needing to touch the original script by hand, reproduces the fit the reviewer's refit is reported to have achieved, recovering the AIC of 19,340.4 that Yee et al. (2025) report for it. What took the original authors a manual refit and a paragraph in the supplement is, for a reader equipped with this schema and its alternative file, a single generation-and-validation step.

4. Application: many-analysts football study

Section 3 assumed an author willing to supply a schema. Most published analyses come with no such thing, so we now ask what the same workflow is worth when none exists. Applied to a manuscript alone, it becomes a reproducibility test: extract a schema from what the paper says, generate and validate code from it, and compare the result against what the paper reports. Agreement means the description was sufficient to regenerate the result. Disagreement means something is missing, ambiguous, or wrong, and is the more informative outcome. We apply this to the many-analysts football study of Silberzahn et al. (2018), which gives 29 independent descriptions of an analysis of one dataset, each with a reported effect size to test against.

Silberzahn et al. (2018) invited 29 research groups to conduct the same data analysis, answering whether referees are more likely to give red cards to dark-skin-toned players than to light-skin-toned players. The provided dataset contains two raters' skin color ratings of players' pictures, the aggregated number of red cards received by a player from a referee across all matches in that player-referee pair, and other relevant player, referee, and game characteristics: player birthday, height, weight, player position, league country, number of victories, ties, and defeats, and number of yellow and red cards received. In Silberzahn et al. (2018), each group first conducted an initial analysis separately, followed by a round-robin peer evaluation. Each group was then given the opportunity to incorporate feedback from the discussion and submitted a final analysis as a report with a formal Method section. All the materials related to the study are publicly available on the Open Science Framework (OSF).

Based on the reports presented by each team, most participating groups adopted a modelling approach to estimate the effect of skin color rating on the likelihood of receiving a red card. As an illustrative example, Team 5 first transformed the data into player–game observations, resulting in a binary response variable for whether a player received a red card, and removed observations with missing values using list-wise deletion. The resulting data were modelled using a generalized linear mixed model (GLMM) with a binomial error distribution. The average skin color rating from two raters was used as the primary covariate, and its estimated coefficient was the main parameter of interest. To account for unobserved heterogeneity, the model included random effects for player and referee, as well as the average skin color rating across referee's country of origin. While Team 5's analysis illustrates one possible approach, other teams may adopt different data preprocessing decisions, distributional assumptions (e.g., linear, lo-

gistic, or Poisson models), or specifications of fixed and random effects, or may conduct the analysis in a Bayesian framework.

Team reports are not straightforward to use as input directly. Beyond the analysis of interest, whether skin color rating predicts red cards, the many-analysts study also includes a second research question and report confidence interval calculations for the estimated effect, neither of which is relevant to the schema extraction or code generation we study here. The report structure adds a further complication: some teams write the final analysis as edits against the initial analysis, for example a covariate dropped, a variable rescaled, a different outcome, rather than as a self-contained methodology section. Recovering a single, self-contained methodology description while setting aside the unrelated research question is its own nontrivial comprehension problem, separate from the task of generating the schema and analysis code that we study here. We therefore pre-extract a simplified methodology text for each team and use this, rather than the original report, as the input for our experiment. The Appendix follows one team through the whole pipeline, giving the pre-extracted text, the schema extracted from it, and the code generated from that schema, along with the prompts used at each stage.

Each team's text is then passed through the workflow of Section 2 unchanged: schema extraction, code generation, and validation.

To accelerate the evaluation of each script, we manually replace, in place, the `lm()`/`glm()` command with `fixest::feols()`/`fixest::feglm()` (Bergé et al., 2026) for generalized linear models, and the `lme4::glmer()` (Bates et al., 2015) function with `glmmTMB::glmmTMB()` (Brooks et al., 2017) for generalized mixed-effects models, after the scripts are generated but before validation. These two functions are in-place replacements of the standard functions for estimating the relevant models, with multi-threading in C++ (Bergé et al., 2026) and Template Model Builder (TMB) (McGillycuddy et al., 2025) for better performance. This replacement should not affect the estimation of the effect size. All LLM-related runs are conducted with Claude Opus 4.5.

Of the 29 teams, we report results for 25. The remaining four (Teams 7, 12, 17, and 18) specify analyses that are too computationally expensive to run inside an iterative validate-and-repair loop, regardless of whether the schema or generated code is correct: Teams 17 and 18 both specify MCMC-based hierarchical models with thousands of random effects, Team 7 a Dirichlet process profile regression, and Team 12 a zero-inflated Poisson regression with several thousand fixed-effect parameters. Across the 25 teams we reproduce, all team scripts were successfully executed after the validation phase. Notably, two scripts ran successfully without any validation, whereas the vast majority reached completion after two or three iterations. The most frequent correction involved resolving broken and overflowing syntax in comments.

Figure 2 shows the comparison of the estimated effect sizes, in odds ratio, from our workflow (reproduced) and those reported in Silberzahn et al. (2018) (reported). The odds ratio is well recovered for most teams, though six show a noticeably larger gap:

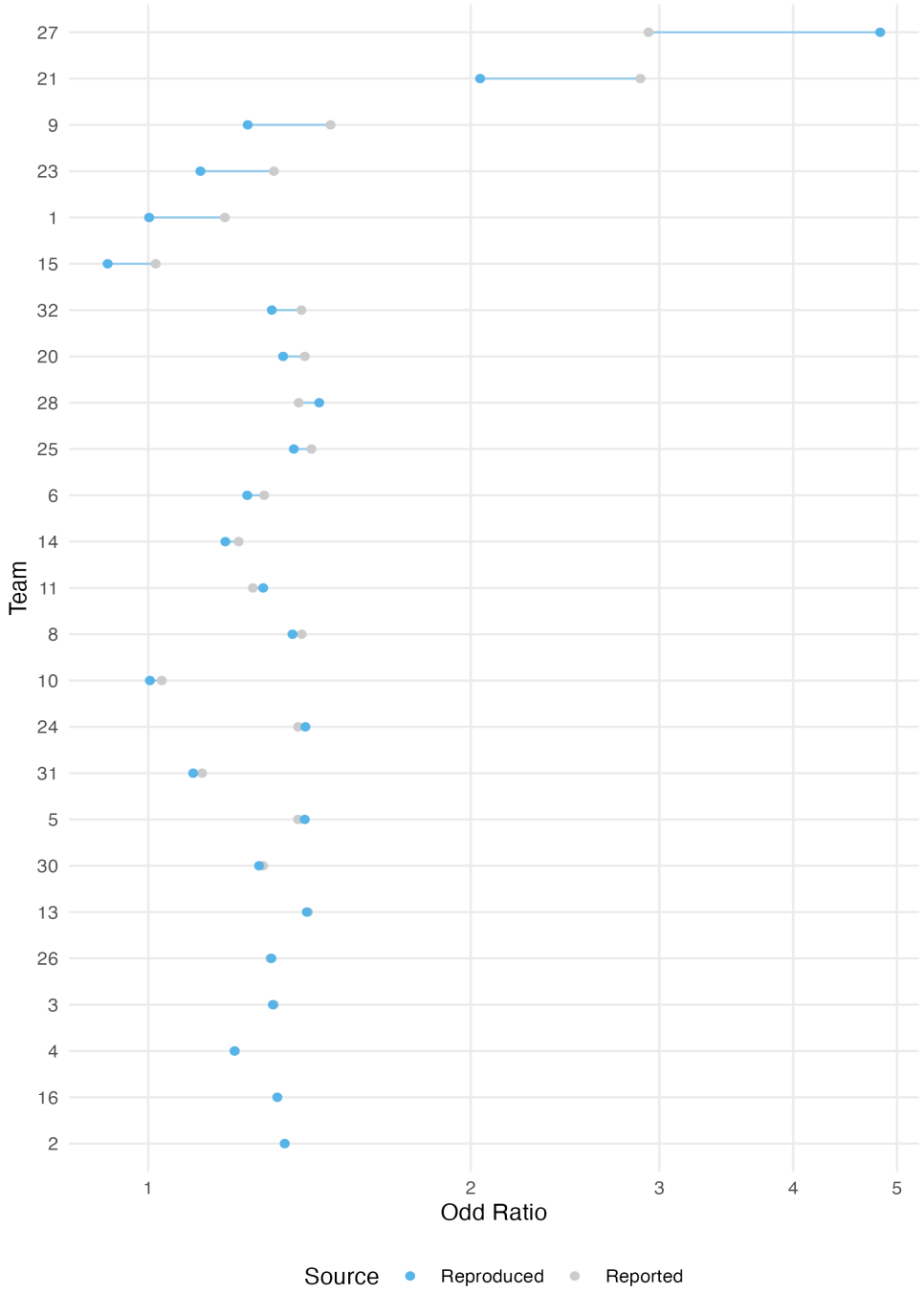


Figure 2. Comparison of the odds ratio of the effect of skin color on receiving red cards in football games from our schema-based reproduction (reproduced) vs. the effect size reported in Silberzahn et al. (2018) (reported). Teams are ordered by the absolute difference between the reproduced and reported odds ratio. The reproduction recovers the reported odds ratio closely for most teams; the largest deviations occur for Team 1, 9, 15, 21, 23, and 27.

Team 1, 9, 15, 21, 23, and 27. Each of these is worth examining, and the schema is what makes the examination possible: because the extracted decisions can be inspected before any code is written, a discrepancy can be attributed to the report, to the code generated from it, or to neither.

Two of the six are properties of the report rather than of our pipeline. Team 9 is the clearest: the text names "referee country" as the covariate, while the reported estimation table lists "player country". The report contradicts itself, and no reading of the manuscript can settle which model produced the published estimate. Team 27 is ambiguous rather than contradictory. The authors write: "In addition to the independent variables specified in the initial approach, … a variable for player position was included, as well as a variable for the league of the player." A statistician might reasonably read this as shorthand for extending the model with interactions, consistent with how the other covariates entered the initial model; read literally, it adds only main effects. The schema records what the text says in both cases, and the discrepancy that follows is the test's finding, not its failure: these are the two reports whose description of the analysis is not sufficient to regenerate the result.

The confidence score ω_i separates these two cases in a way worth noting. For Team 27, extraction returned `create_interaction_terms` with medium confidence, flagging the step whose reading is in doubt before any code was written. For Team 9, every step came back with high confidence, which is the correct response rather than an oversight: the report's text is unambiguous on its own terms, and the inconsistency appears only when that text is set against the estimation table. Since ω_i records the model's confidence that the schema reflects the source text, it can surface a description that is ambiguous, but not one that is internally clear and contradicted elsewhere. Catching the latter requires comparing the schema against the reported results, which is what the test as a whole does.

Teams 1 and 23 are failures of a different kind, and they are ours: the schema is right and the code is wrong. Both require reshaping the data from player-referee pairs to individual games, disaggregating win/loss/draw counts into game-level records. The schema correctly specifies this step, but the generated code fails to execute it as intended, and the resulting error passes through validation silently rather than triggering a repair.

Team 15's discrepancy is unrelated to either stage: it comes down to a difference in default contrast coding between R (our reproduction) and SPSS (the original analysis) — SPSS's effect coding compares each level to the grand mean, while R's default dummy coding compares to a reference level. Setting the `contrast` option in R to match recovers the reported odds ratio exactly, so this reflects a software convention rather than a modeling or LLM error.

That leaves Team 21, whose discrepancy we have been unable to explain. Examining both the extracted schema and the generated code turned up nothing that accounts for the gap, and we report it as unresolved rather than assign it to a cause we cannot establish; the procedure localizes most discrepancies, but evidently not all of them.

Setting Team 21 aside, the five remaining discrepancies divide into two whose reports cannot be resolved to a single analysis, two where our code generation failed, and one that reflects a difference in software convention. Only the middle pair are errors in the workflow itself; the first two are what the test was built to surface.

4.1. *Exploring alternatives*

During the modelling process, researchers may consider a range of alternative specifications, such as different covariate and random effects structures, before arriving at a final model. This can be seen as a set of alternative decisions within the same modelling step, which fits naturally into the schema workflow described in Section 2.2. As an example, we return to Team 5, whose analysis was described above. Having settled on a binomial GLMM with a logit link for the probability that a player receives a red card in a single game, Team 5 tests three random effect structures:

$$\textbf{Model 1 (gm1):} \quad \begin{aligned} \operatorname{logit}(p_{ijk}) &= \beta_0 + \beta_1 X_{ijk} + u_j + v_k \\ u_j &\sim N(0, \sigma^2_{\text{player}}), \quad v_k \sim N(0, \sigma^2_{\text{ref}}) \end{aligned}$$

$$\textbf{Model 2 (gm2):} \quad \begin{aligned} \operatorname{logit}(p_{ijk}) &= \beta_0 + \beta_1 X_{ijk} + u_j + v_{0k} + v_{1k} X_{ijk} \\ u_j &\sim N(0, \sigma^2_{\text{player}}), \quad \mathbf{v}_k \sim N\left(\mathbf{0}, \Sigma_{\text{ref}}\right) \end{aligned}$$

$$\textbf{Model 3 (gm3):} \quad \begin{aligned} \operatorname{logit}(p_{ijkm}) &= \beta_0 + \beta_1 X_{ijkm} + u_j + v_k + w_{0m} + w_{1m} X_{ijkm} \\ u_j &\sim N(0, \sigma^2_{\text{player}}), \quad v_k \sim N(0, \sigma^2_{\text{ref}}), \quad \mathbf{w}_m \sim N\left(\mathbf{0}, \Sigma_{\text{country}}\right) \end{aligned}$$

where p denotes the probability of a player receiving a red card, and X represents the player's skin tone rating. The subscripts i, j, k, and m index the individual observation, player, referee, and the referee's home country, respectively. The parameter β_0 represents the global baseline log-odds of receiving a red card, and β_1 estimates the fixed effect of a player's skin tone. The random effects include

- a random intercept for players, $u_j \sim N(0, \sigma^2_{\text{player}})$, in Models 1, 2, and 3,
- a random intercept for referees, $v_k \sim N(0, \sigma^2_{\text{ref}})$, in Models 1 and 3, plus an additional random slope for referees in Model 2, and
- a random intercept and slope for referee countries, $\mathbf{w}_m \sim N\left(\mathbf{0}, \Sigma_{\text{country}}\right)$, in Model 3.

All three are specified in a single YAML file, following the format described in Section 2.2 and shown in Listing 8. Merging it with Team 5's schema produces one schema per alternative, and each is passed through the same code generation and validation used for the original, yielding three validated scripts from one file and no hand-editing of the team's code. The estimated effect of skin tone is unchanged across this expansion,

matching the value Team 5 report and shown for this team in Figure 2.

Listing 8 Example of a YAML file specifying alternatives for the random effects structure in the many-analysts analysis of Team 5.

```yaml
meta:
  type: alternatives
  branch: multi
nodes:
- overrides: specify_random_effects_structure
  alternatives:
  - id: gm1
    decision: a random effect of player and referee
    rationale: ''
  - id: gm2
    decision: a random effect of player and referee, and a random effect of
    ↪  skin-tone across referees
    rationale: ''
  - id: gm3
    decision: a random effect of player, referee, and referees' countries of
    ↪  origin, and a random effect of skin tone across referees' countries of
    ↪  origin
    rationale: ''
```

5. Discussion

Our results highlight a persistent gap between computational reproducibility and analytical understandability in modern data analysis workflows. Current reproducibility practices ensure that published results can be regenerated from shared data and code, but as analyses grow in complexity and involve iterative decision-making, they fall well short of making those analyses understandable to readers. Our proposed schema addresses this gap by introducing an intermediate representation that explicitly encodes analytical decisions to bridge between narrative descriptions and implementation details. When coupled with LLMs, this representation enables the extraction of analytical structure from written reports, its translation into analysis scripts, and the exploration of alternative analytical choices.

Our approach differs in this respect from recent work that builds fully automated systems using multiple LLM agents to plan, generate, execute, and refine data analyses (Hu et al., 2024; Jansen et al., 2025). These systems rely on the same underlying capability of translating analytical specifications into code that our workflow uses, but they aim to automate the analytical decisions themselves, not just their translation into code. The failure modes we document in Section 4 suggest this is premature: even the comparatively narrower task of translating an already-specified decision into working code is not yet fully reliable. We take this as support for our position that the harder and more consequential part of a data analysis is the sequence of decisions a researcher

makes, not the code that implements them, and that this part should remain explicit and in the hands of the analyst rather than be automated away.

Where no schema is supplied, the same workflow serves as a reproducibility test, and we apply it in this form to the many-analysts football study. It surfaces inconsistencies and ambiguities in the original reports that computational reproducibility alone would not catch: Team 9's report names one covariate in its text and a different one in its estimation table, a contradiction that is invisible if one only re-runs the authors' code, and that only becomes apparent once the described analysis is written down as an explicit set of decisions and executed. Examining the largest discrepancies also separates defects in the reports from failures of our own pipeline — two of each, with a fifth attributable to a difference in software convention. The separation is possible because the schema sits between the manuscript and the code and can be checked against the manuscript on its own, and it matters for where effort should be directed: an unresolvable report calls for disambiguation at the source, and the confidence score ω_i is the natural hook for it, having flagged the disputed step in Team 27's report before any code was written, whereas a code-generation failure calls for improvements to the validation stage itself.

A caveat applies to the failure modes we report: they describe the behaviour of one model at one point in time. Current models parse methodology text reliably for the most part, and where they slip is on statistical nuance that a trained reader supplies without thinking. The phrase "a random effect of referee number across average rating" would be read by most statisticians as implying both a random intercept and a random slope; read literally, as the model reads it, it specifies a random slope only. This points to a gap between current models and human analysts in specialized statistical knowledge (Sun et al., 2026), but given the pace at which these models are improving we expect this particular gap to narrow, and we do not regard it as a structural limitation of the workflow. What the workflow provides, independently of model capability, is somewhere to catch such a misreading: the schema is inspectable before any code is generated, so a misread decision is visible to the analyst as a sentence rather than buried in a script.

Our validate-and-repair loop assumes each candidate script can be executed and diagnosed within an interactive timeframe. This assumption does not hold for analyses that are too computationally expensive to run inside such a loop (Teams 7, 12, 17, and 18 in the many-analysts football study). As a future direction, this could be addressed by implementing a background job queue to allow longer-running scripts to be executed asynchronously.

Turning to the other stage, the validation loop successfully corrects a range of implementation issues, including errors caused by overflowing comments and other syntax problems. What it does not catch are transformations that run without error but do not do what the schema asked, of which the reshaping failures behind Teams 1 and 23 are the clearest instance: changing the unit of analysis from player-referee dyads to individual games produces code that executes and returns an estimate regardless of whether the disaggregation was done correctly. We also observe some difficulties in

parsing birthdates. Since the LLM does not have access to the underlying raw data, due to privacy constraints, it must rely on trial and error when inferring the correct date format, in this case "DD-MM-YYYY". This suggests that certain types of data analysis tasks may not be fully automatable in settings where the model cannot directly inspect the data, even if the data is theoretically available for computation.

While the schema requires a rationale ρ_i for each decision, nothing in the format itself can guarantee that this rationale is substantive rather than perfunctory; an analyst can satisfy the schema with a low-effort entry in the same way that a required code comment or commit message can be filled with boilerplate. The schema makes recording a decision's rationale cheap and structured, but, as with any documentation requirement, cannot enforce that authors use it thoughtfully. It remains an open question whether resolving $\mathcal{I}_i$ and $\mathcal{O}_i$ to concrete dataset variables ahead of time, rather than leaving this mapping for the LLM to infer at the point of code generation, meaningfully improves the reliability of the resulting code.

The reproducibility-understandability issue considered in this work is still at an early stage, and some software design choices are worth examining. We currently implement a prototype as an R package, although the understandability problem is broadly applicable to statistical analyses and is not associated with any particular programming language. Hence, we envision the implementation eventually evolving into a general purpose tool, such as a command-line utility, that can support analyses across different programming languages. In the current implementation, analytical decisions are recorded in a single YAML file. An alternative would be to record individual decisions in separate Markdown files. While YAML files provide an explicit structure for machine readability, Markdown files offer greater flexibility for documenting decisions. Recording decisions in separate files would also allow them to be linked across time, showing how decisions evolve over the course of an analysis. This could in turn support a broader research-management system, provided that decisions are properly attributed to individual researchers and that a record of the changes leading to the final analysis is maintained.

6. Acknowledgments

The implementation of the schema is available on CRAN as an R package, `tines`. The source code for reproducing the work reported in this paper is available in the Supplementary Materials and can be found at: https://github.com/huizezhang-sherry/paper-schema.

All analyses were carried out in R (R Core Team, 2026), using the `tines`, `tidyverse` (Wickham et al., 2019), `VGAM` (Yee, 2025), `VGAMdata` (Yee & Gray, 2025), `lme4` (Bates et al., 2015), `glmmTMB` (Brooks et al., 2017; McGillycuddy et al., 2025), and `fixest` (Bergé et al., 2026) packages. The article is created using Quarto (Allaire et al., 2026).

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C., & Woodhull, G. (2026). *Quarto* (Version 1.9) [Computer software]. https://doi.org/10.5281/zenodo.5960048

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Bergé, L. R., Butts, K., & McDermott, G. (2026). Fast and user-friendly econometrics estimations: The R package fixest. *arXiv Preprint arXiv:2601.21749*. https://doi.org/10.48550/arXiv.2601.21749

Bertran, M., Fogliato, R., & Wu, Z. S. (2026). *Many AI Analysts, One Dataset: Navigating the Agentic Data Science Multiverse*. arXiv. https://doi.org/10.48550/arXiv.2602.18710

Brooks, M. E., Kristensen, K., van Benthem, K. J., Magnusson, A., Berg, C. W., Nielsen, A., Skaug, H. J., Maechler, M., & Bolker, B. M. (2017). glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R Journal*, *9*(2), 378–400. https://doi.org/10.32614/RJ-2017-066

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. *arXiv Preprint arXiv:2107.03374*. https://arxiv.org/abs/2107.03374

Gao, R., & Xiao, S. C. (2026). *Nonstandard Errors in AI Agents*. arXiv. https://doi.org/10.48550/arXiv.2603.16744

Hu, S., Lu, C., & Clune, J. (2024, October). *Automated Design of Agentic Systems*. https://openreview.net/forum?id=t9U3LW7JVX

Huang, J., Wang, C., Zhang, J., Yan, C., Cui, H., Inala, J. P., Clement, C., & Duan, N. (2022). Execution-based Evaluation for Data Science Code Generation Models. In E. Dragut, Y. Li, L. Popa, S. Vucetic, & S. Srivastava (Eds.), *Proceedings of the Fourth Workshop on Data Science with Human-in-the-Loop (Language Advances)* (pp. 28–36). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.dash-1.5

Jansen, J. A., Manukyan, A., Khoury, N. A., & Akalin, A. (2025). Leveraging large language models for data analysis automation. *PLOS ONE*, *20*(2), e0317084. https://doi.org/10.1371/journal.pone.0317084

Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2026). A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, *35*(2), 1–72.

Jun, E., Birchfield, M., De Moura, N., Heer, J., & Just, R. (2022). Hypothesis formalization: Empirical findings, software limitations, and design implications. *ACM Transactions on Computer-Human Interaction (TOCHI)*, *29*(1), 1–28.

Jun, E., Seo, A., Heer, J., & Just, R. (2022). *Tisane: Authoring statistical models via formal reasoning from conceptual and data relationships. CHI '22*, 116. https://doi.org/10.1145/3491102.3501888

Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., Yih, W.-T., Fried, D., Wang, S., & Yu, T. (2023). DS-1000: A Natural and Reliable Benchmark for

Data Science Code Generation. *Proceedings of the 40th International Conference on Machine Learning*, 18319–18345. https://proceedings.mlr.press/v202/lai23b.html

Luo, A., Du, J., Tian, F., Xian, X., Specht, R., Wang, G., Bi, X., Fleming, C., Srinivasa, J., Kundu, A., Hong, M., & Ding, J. (2025). Can Agentic AI Match the Performance of Human Data Scientists? *2025 IEEE 10th International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP)*, 206–210. https://doi.org/10.1109/CAMSAP66162.2025.11423949

McGillycuddy, M., Warton, D. I., Popovic, G., & Bolker, B. M. (2025). Parsimoniously fitting large multivariate random effects in glmmTMB. *Journal of Statistical Software*, *112*(1), 1–19. https://doi.org/10.18637/jss.v112.i01

Peng, R. D. (2011). Reproducible Research in Computational Science. *Science*, *334*(6060), 1226–1227. https://doi.org/10.1126/science.1213847

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://doi.org/10.32614/R.manuals

Sarma, A., Kale, A., Moon, M. J., Taback, N., Chevalier, F., Hullman, J., & Kay, M. (2023). Multiverse: Multiplexing alternative data analyses in r notebooks. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3544548.3580726

Shah, S. M. H., Hopfgartner, F., & Bleier, A. (2026). *Automating Computational Reproducibility in Social Science: Comparing Prompt-Based and Agent-Based Approaches*. https://doi.org/10.1145/3774905.3795485

Silberzahn, R., Uhlmann, E. L., Martin, D. P., Anselmi, P., Aust, F., Awtrey, E., Bahník, Š., Bai, F., Bannard, C., Bonnier, E., Carlsson, R., Cheung, F., Christensen, G., Clay, R., Craig, M. A., Dalla Rosa, A., Dam, L., Evans, M. H., Flores Cervantes, I., … Nosek, B. A. (2018). Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results. *Advances in Methods and Practices in Psychological Science*, *1*(3), 337–356. https://doi.org/10.1177/2515245917747646

Simonsohn, U., Simmons, J. P., & Nelson, L. D. (2020). Specification curve analysis. *Nature Human Behaviour*, *4*(11), 1208–1214. https://doi.org/10.1038/s41562-020-0912-z

Steegen, S., Tuerlinckx, F., Gelman, A., & Vanpaemel, W. (2016). Increasing transparency through a multiverse analysis. *Perspectives on Psychological Science*, *11*(5), 702–712. https://doi.org/10.1177/1745691616658637

Sun, M., Han, R., Jiang, B., Qi, H., Sun, D., Yuan, Y., & Huang, J. (2026). LAMBDA: A Large Model Based Data Agent. *Journal of the American Statistical Association*, *121*, 1–13. https://doi.org/10.1080/01621459.2025.2510000

United Nations Development Programme. (2025). *Human development report 2025*. United Nations Development Programme. https://hdr.undp.org/

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wrobel, J., Hector, E. C., Crawford, L., McGowan, L. D., Silva, N. da, Goldsmith, J., Hicks, S., Kane, M., Lee, Y., Mayrink, V., Paciorek, C. J., Usher, T., & Wolfson, J. (2024). Partnering With Authors to Enhance Reproducibility at JASA. *Journal of the American Statistical Association*, *119*(546), 795–797. https://doi.org/10.1080/01621459.2024.2340557

Yee, T. W. (2025). *VGAM: Vector generalized linear and additive models*. https://CRAN.R-project.org/package=VGAM

Yee, T. W., Frigau, L., & Ma, C. (2025). Heaping and seeping, GAITD regression and doubly constrained reduced-rank vector generalized linear models in smoking studies. *The Annals of Applied Statistics*, *19*(4), 3045–3070.

Yee, T. W., & Gray, J. (2025). *VGAMdata: Data supporting the VGAM package*. https://CRAN.R-project.org/package=VGAMdata